

PHASE 5 — PROJECT DEVELOPMENT PHASE

Project Development Phase

Codebase Implementation, Module Structure & Feature Delivery

Development documentation for an AI-powered debt relief and financial recovery platform — covering the delivered backend and frontend codebase, core module structure, and the functional features implemented during the build.

1. Codebase Structure

This document records the delivered, production-ready codebase for the AI-powered debt relief and financial recovery platform, organized into a Python/FastAPI backend and a React/Vite frontend. The structure below reflects the final folder layout at the close of the development phase.

1.1 Backend (backend/)

Python · FastAPI · SQLite (local) · SQLAlchemy ORM

`app/main.py` — API routes for JWT authentication, user profiles, loan management, and AI letter generation.

`app/models.py` — relational SQLAlchemy database models mapping to the SQLite/Postgres tables.

`app/schemas.py` — Pydantic validation schemas.

`app/auth.py` — authentication utilities using direct bcrypt password hashing and JWT encoding/decoding.

`app/services/financial_engine.py` — calculators for DTI ratios, EMI ratios, surplus cash flow, and rule-based predictions.

`app/services/gemini_service.py` — Google Gemini API client for hardship letter compilation, with a local mockup fallback.

`tests/` — pytest automated unit tests verifying calculator boundaries.

`requirements.txt` — Python dependency list.

`run.py` — FastAPI server startup script.

1.2 Frontend (frontend/)

React.js (Vite) · Vanilla CSS

`src/index.css` — design-token styling: responsive grids, HSL variable system, glow accents, glassmorphic cards, keyframe animations.

`src/context/AuthContext.jsx` — session state, local tokens, user registration, and dynamic API URL lookups.

`src/pages/Login.jsx`, `Register.jsx` — credential management views.

`src/pages/Dashboard.jsx` — summary cards, loan tables, CRUD modals, and the SVG distress-score gauge.

`src/pages/SettlementPredictor.jsx` — target settlement amounts, warning badges, comparison bars, lump-sum vs. installment plans.

`src/pages/LetterGenerator.jsx` — AI letter generation inputs, bulleted strategies, copy/download actions.

`src/pages/History.jsx` — accordion logs of past generated letters.

`vercel.json` — SPA routing rewrite rules for Vercel edge deployment.

2. Runtime Request Flow

The diagram below traces a single request end-to-end through the delivered codebase — from a frontend page component down to the persistence layer, showing where the Gemini AI service and its local fallback sit relative to the deterministic financial engine.

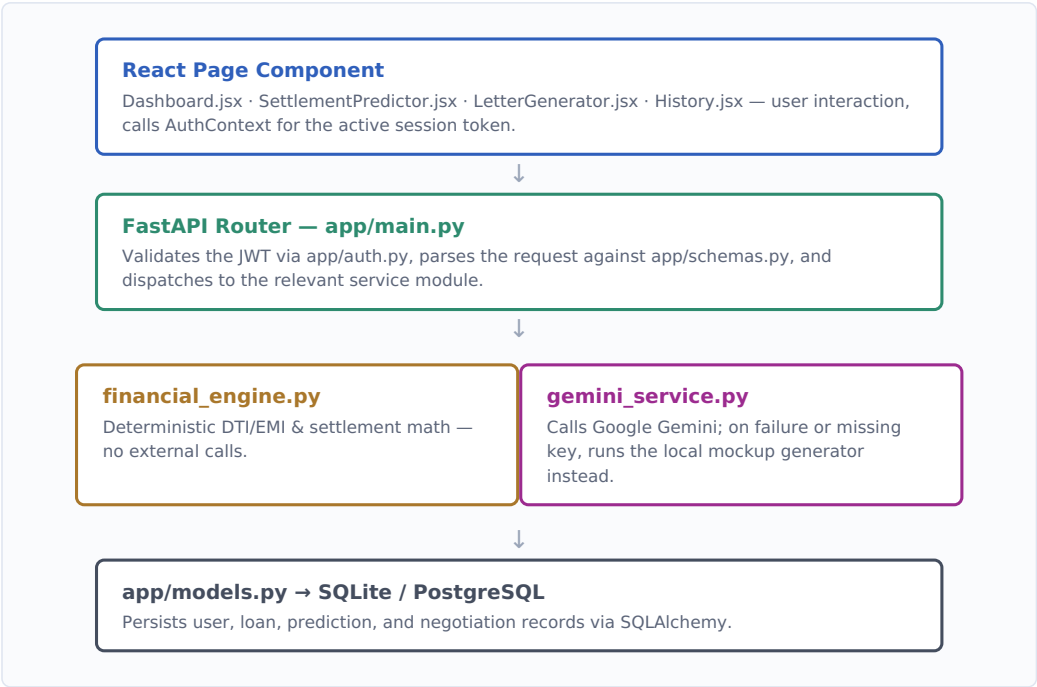


Figure 1. Request path from a React page through the FastAPI router, service layer, and persistence layer.

2.1 Why the fallback matters

gemini_service.py is built as a wrapper, not a hard dependency: every call path checks for a valid Gemini API key and a successful response before falling back to the rule-based local mockup generator. This keeps letter generation from throwing server errors during API outages, rate limits, or local development without credentials — the same behavior flagged as a risk mitigation in Phase 4.

3. Core Functional Features Implemented

Eight core features were implemented and verified during this phase, spanning authentication, financial analysis, AI drafting, and audit history.

Feature	Description
1. JWT Authentication	Secure login and registration with hashed password persistence.
2. Interactive Dashboard	Real-time evaluation of total debt liabilities, gross income, and basic living expenses.
3. Visual Stress Meter	Custom animated dial reflecting the borrower's debt distress score based on calculated DTI and EMI ratios.
4. Adaptive Settlement Predictor	Suggests realistic target settlement percentages (35%-85%) based on delinquency duration, loan type (secured vs. unsecured), and financial stress.
5. Structured Payoff Plans	Breaks predicted settlement values down into an immediate lump-sum payoff vs. structured 3-month or 6-month payment schedules.
6. AI Negotiation Letter Generator	Triggers the Google Gemini API to draft professional, personalized hardship letters from input parameters and custom text.
7. Auto Fallback Engine	If the Gemini API key is missing or the service is offline, the local mock generator runs automatically, compiling letters without crashing the server.
8. Audit Logs & History	Expandable accordion log listing all past proposal documents for a given user.

3.1 Feature-to-Module Traceability

Feature	Primary Backend Module	Primary Frontend Page
JWT Authentication	app/auth.py	Login.jsx / Register.jsx
Dashboard & Stress Meter	services/financial_engine.py	Dashboard.jsx
Settlement Predictor	services/financial_engine.py	SettlementPredictor.jsx
AI Letter Generator & Fallback	services/gemini_service.py	LetterGenerator.jsx
Audit Logs & History	app/models.py (ai_history)	History.jsx

4. Additional Notes & Recommendations

The following points are not part of the original delivery notes, but are common considerations worth tracking as the codebase moves from development into hardening and release.

4.1 Code Quality & Testing Coverage

- ✓ pytest suite currently verifies financial_engine.py boundary cases (Low → Severe stress); extend coverage to gemini_service.py's fallback branch specifically.
- ✓ Add integration tests that exercise the full router → service → database path, not just unit-level calculator tests.
- ✓ Add frontend component tests (e.g. Vitest + React Testing Library) for the SVG stress gauge and the accordion history view, which currently only have manual smoke tests.
- ✓ Wire test runs into CI so pytest and npm run build both gate merges to main, matching the Definition of Done from Phase 4.

4.2 Security & Compliance Considerations

- Confirm bcrypt hashing parameters (work factor) are tuned for production load, not just local defaults.
- Ensure JWT secrets and the Gemini API key are read only from environment variables in run.py / config, never hard-coded in committed source.
- Apply the row-level UserID checks recommended in Phase 3 consistently across every new endpoint added in app/main.py.
- Redact or mask financial figures and letter content in any application logs generated by gemini_service.py.

4.3 Deployment Readiness

Area	Status	Note
Backend build	Ready	run.py starts the FastAPI/Uvicorn server; environment variables externalized.
Frontend build	Ready	npm run build verified; vercel.json handles SPA route rewrites.
Database migration path	Pending	Alembic not yet wired in; still a direct SQLAlchemy create_all() on SQLite.
Monitoring & alerting	Pending	No uptime or Gemini-usage dashboard yet — see Suggested Next Steps.

4.4 Suggested Next Steps

Phase 6 — Post-launch monitoring

Phase 7 — User feedback loop

Phase 8 — v2 roadmap

Ongoing — Sprint retrospectives

- **Phase 6:** Post-launch monitoring plan — uptime checks, error logging, and Gemini API usage dashboards.
- **Phase 7:** User feedback loop — a lightweight in-app survey or feedback form after letter generation.
- **Phase 8:** Roadmap for v2 features (e.g. multi-lender bulk negotiation, email delivery of letters).
- **Ongoing:** Sprint retrospectives after each cycle to capture velocity and refine future sprint sizing.